A Linguagem $lang^2$

Neste documento é apresentada a especificação da linguagem de programação que será usada na implementação dos trabalhos da disciplina CSI506-Compiladores, denominada $lang^2$. A linguagem $lang^2$ tem propósito meramente educacional, contendo construções que se assemelham às de várias linguagens conhecidas. No entanto, a linguagem não é um subconjunto de nenhuma delas.

A descrição da linguagem é dividida entre seus diversos aspectos, a saber: sintático, descrito na Seção 1, e léxico, descrito na Seção 2. Este documento ainda não define formalmente a semântica de $lang^2$.

1 Estrutura Sintática

Em linhas gerais, um programa nesta linguagem é constituído por um conjunto de definições de tipos de dados, seguido por definições de funções. A estrutura sintática da linguagem é dividida em: *tipos de dados e declarações*, *funções*, *comandos* e *expressões*. Cada uma dessas estruturas é detalhada nas subseções subsequentes. A gramática completa da linguagem $lang^2$ pode ser encontrada na Seção 5.

1.1 Tipos de Dados e Declarações

Programas $lang^2$ podem conter definições de tipos de dados do tipo registro, os quais são definidos usando a palavra-chave **data**. Após a palavra-chave **data**, segue o nome do novo tipo, o qual deve começar com uma letra maiúscula, seguido por uma lista de declarações de variáveis delimitada por chaves. Por exemplo, um tipo para representar uma fração pode ser definido como:

```
data Racional {numerador :: Int;
               denominador :: Int;
              }
```

Esse tipo é denominado *Racional* e contém dois atributos do tipo inteiro, denominados *numerador* e *denominador*. A sintaxe para especificar os atributos de um tipo registro é a mesma usada para declarações de variáveis em funções, isto é, o nome do atributo ou variável seguido por dois pontos, o tipo do atributo (ou variável) e finalizado com ponto e vírgula.

1.2 Funções

A definição de funções e procedimentos é feita fornecendo-se o nome da função (ou procedimento) e sua lista de parâmetros. Após os parâmetros, segue o símbolo *::* e a anotação de tipo da função. Na anotação, representamos o tipo da função com a mesma notação usada na linguagem Haskell, isto é, *Tipo1 -> Tipo2 -> ... -> Tipo de retorno*. Note que uma função pode ter mais de um valor de retorno, os quais são separados pelo símbolo &. Para procedimentos, que não retornam valores, o tipo é anotado como *Void*. Por fim, segue o bloco de comandos.

```
main :: Void {
  print(str(fat(10)[0]));
}

fat num :: Int {
  if (num < 1)
    return 1;
  else
    return num * fat(num-1)[0];
}

divmod num div :: Int -> Int -> Int & Int {
  q = num / div;
  r = num % div;
  return q r;
}
```

Figura 1: Exemplos de funções e procedimentos na linguagem $lang^2$.

Um exemplo de programa na linguagem $lang^2$ é apresentado na Figura 1, contendo a definição de um procedimento denominado *main* e das funções *fat* e *divmod*. A função *fat* recebe um valor inteiro como parâmetro e retorna o fatorial desse valor. A função *divmod* é um exemplo de função com mais de um valor de retorno, recebendo dois parâmetros inteiros e retornando o quociente e o resto da divisão do primeiro pelo segundo.

1.3 Classes e Instâncias de Tipos

A linguagem $lang^2$ permite sobrecarga de nomes por meio de classes de tipos, que funcionam de forma análoga às da linguagem Haskell. Uma classe define apenas uma coleção de nomes e seus respectivos tipos, sendo análoga a uma interface na linguagem Java. Além disso, uma classe de tipo é sempre parametrizada por uma variável de tipo. Cada instância da classe deve substituir essa variável por um tipo concreto e prover todas as definições da interface com um corpo apropriado.

Na Figura 2, apresentamos um programa que define a classe **Chr** com o parâmetro de tipo a. Nessa classe, definimos apenas o símbolo **tychr**, cujo tipo representa uma função que mapeia o parâmetro de tipo a para um valor do tipo **Char**. Em seguida, definimos duas instâncias, uma para o tipo **Int** e outra para o tipo **Bool**. A declaração **instance Chr for Int** permite implementar a classe **Chr** para o tipo **Int**, substituindo o parâmetro de tipo a por **Int**. No corpo da função **main**, utilizamos o método **tyChr** com o mesmo nome para dois tipos distintos.

1.4 Comandos

A linguagem $lang^2$ apresenta apenas seis comandos básicos, classificados em comandos de atribuição, seleção, retorno, iteração e chamada de funções e procedimentos.

O comando de atribuição possui sintaxe semelhante à das linguagens imperativas C/C++ e Java, na qual uma expressão do lado esquerdo especifica o endereço onde será armazenado o valor resultante da avaliação da expressão do lado direito. A linguagem apresenta dois comandos

```
class Chr a {
    tychr :: a -> Char;
}
instance Chr for Int {
     tychr v :: Int -> Char {
        return 'I';
     }
}
instance Chr for Bool {
     tychr v :: Bool -> Char {
        return 'B';
     }
}
main :: Void {
  print(tyChr(0)[0]);
  print(tyChr(true)[0]);
}
```

Figura 2: Exemplos de classes de tipos e instâncias na linguagem $lang^2$.

de seleção: *if-then* e *if-then-else*. Os valores de retorno de uma função são definidos por meio do comando **return**, seguido por uma lista de expressões separadas por espaço.

A linguagem $lang^2$ apresenta apenas um comando de iteração, com as seguintes estruturas:

```
iterate (expr) cmd
iterate (i : expr) cmd
```

O comando **iterate** especifica um trecho de código que será executado uma quantidade de vezes determinada pela avaliação da expressão delimitada por parênteses. Ressalta-se que a expressão é avaliada **uma única vez**, e o laço só será executado se o valor resultante for maior que zero. A segunda forma do comando **iterate** permite o uso de uma variável contadora i, que pode ser utilizada dentro do corpo do laço. Caso a expressão seja um vetor, o comando itera sobre o número de elementos do vetor na primeira forma, enquanto, na segunda forma, a variável contadora assume cada elemento do vetor.

Chamadas de funções e procedimentos são consideradas comandos. A sintaxe para chamada de procedimento consiste no nome do procedimento seguido por uma lista de expressões separadas por espaço e delimitadas por parênteses. Em $lang^2$, os parênteses não devem ser separados do nome da função por espaço. Por exemplo, a chamada ao procedimento *main* da Figura 1 é:

```
main();
```

A chamada de função é semelhante; no entanto, deve-se especificar uma lista de endereços para armazenar os valores de retorno, como na seguinte chamada da função *divmod*:

```
divmod(5 2)<q r>;
```

Esse comando define que os valores de retorno da função serão atribuídos às variáveis q e r.

Por fim, um bloco de comandos é definido como uma sequência de zero ou mais comandos delimitada por chaves.

1.5 Expressões

Expressões são abstrações sobre valores e, em $lang^2$, são muito semelhantes as expressões aritméticas de outras linguagens, i.e., possuem os operadores aritméticos usuais(+,-,*,/,%) além de operadores lógicos (&&, !), de comparação (<,==,!=) e valores (inteiros, caracteres, booleanos, floats, registros, vetores e chamadas de métodos). Adicionalmente, parêntesis podem ser usados para determinar a **prioridade** de uma sub-expressão.

Observe, no entanto, que o conjunto de operadores é reduzido. Por exemplo, operações com valores lógicos (tipo booleano) são realizadas com os operadores de conjunção (&&) e negação (!). A linguagem não prover operadores para as demais operações lógicas. Como consequência, se queremos realizar uma seleção quando o valor de ao menos uma de duas expressões, p e q, resulta em verdadeira, escrevemos:

```
1 if(!(!p && !q)) { ... }
```

As chamadas de funções são expressões. Porém, diferentemente das linguagens convencionais, usa-se um índice para determinar qual dos valores de retorno da função será usado. Assim, a expressão $divmod(52)[0]$ se refere ao primeiro retorno, enquanto a expressão $divmod(52)[1]$ ao segundo. Note que a indexação dos retornos da função é feita de maneira análoga ao acesso de vetores, no qual o primeiro retorno é indexado por 0, o segundo por 1 e assim sucessivamente.

A expressão $x * x + 1 < fat(2 * x)[0]$ contém operadores lógicos aritméticos e chamadas de funções. Porém, em qual ordem as operações devem ser realizadas? Se seguirmos a convenção adotada pela aritmética, primeiramente deve ser resolvidas as **operações mais fortes** ou de **maior precedência**, i.e. a multiplicação e a divisão, seguida das **operações mais fracas** ou de **menor precedência**, i.e. a soma e subtração. Assim certos operadores tem prioridade em relação a outros operadores, i.e., devem ser resolvidos antes de outros. Para a expressão $x*x+1$ é fácil ver que a expressão $x * x$ deve ser resolvida primeiro e em seguida deve-se somar 1 ao resultado. E quando há operadores de tipos diferentes, como na expressão $x*x+1 < fat(2*x)[0]$? A situação é semelhante, resolve-se o que tem maior precedência, a qual é determinada pela linguagem. Neste exemplo, a última operação a ser realizada é a operação de comparação, cuja precedência é a menor dentre todos os operadores da expressão. A Tabela 1 apresenta a precedência dos operadores da linguagem $lang^2$. O operador que tiver o maior valor da coluna *nível* tem maior precedência.

Sabendo a precedência dos operadores podemos determinar em qual ordem as operações devem ser executadas quando há operadores com diferentes níveis de precedência. Entretanto, como determinar a ordem das operações se uma determinada expressão contém diferentes operadores com a mesma precedência, como nas expressões $v[3].y[0]$ e $x/3 * y$?

Em situações como essas, determinamos a ordem de avaliação das operações a partir da associatividade dos operadores, que pode ser à esquerda ou à direita. Quando os operadores são associativos à esquerda, resolvemos a ordem das operações da esquerda para a direita. Caso os operadores sejam associativos à direita, fazemos o inverso. Em ambas as expressões $v[3].y[0]$ e $x/3 * y$, os operadores são associativos à esquerda. Portanto, na primeira expressão, primeiro é realizado o acesso ao vetor v, depois acesso ao membro y e, por fim, acesso ao vetor de y. Na segunda expressão, realiza-se primeiro a divisão de x por 3 e, em seguida, a multiplicação do resultado por y.

Nível	Operador	Descrição	Associatividade
7	[ ] . ( )	acesso a vetores acesso aos registros parêntesis	esquerda
6	! -	negação lógica menos unário	direita
5	* / %	multiplicação divisão resto	esquerda
4	+ -	adição subtração	
3	<	relacional	não é associativo
2	== ! =	igualdade diferença	esquerda
1	&&	conjunção	esquerda

Tabela 1: Tabela de associatividade e precedência dos operadores. Tem a maior precedência o operador de maior nível.

2 Estrutura Léxica

A linguagem usa o conjunto de caracteres da tabela ASCII[1]. Cada uma das possíveis categorias léxicas da linguagem são descritas a seguir:

- Um **identificador (ID)** é uma sequência de letras, dígitos e sobrescritos (*underscore*) que, obrigatoriamente, começa com uma letra minúscula. Exemplos de identificadores: *var*, *var_1* e *fun10*;
- Um **nome de tipo (TYID)** é semelhante a regra de identificadores, porém a primeira letra é maiúscula; Exemplos de nomes de tipos: *Racional* e *Point*;
- Um **literal inteiro (INT)** é uma sequência de um ou mais dígitos;
- Um **literal ponto flutuante (FLOAT)** é uma sequência de zero ou mais digitos, seguido por um ponto e uma sequência de um ou mais digitos. Exemplos de literais ponto flutuante: 3.141526535, 1.0 e .12345;
- Um **literal caractere (CHAR)** é um único caractere delimitado por aspas simples. Os caracteres especiais quebra-de-linha, tabulação, *backspace* e *carriage return* são definidos usando os caracteres de escape \n, \t, \b e \r, respectivamente. Para especificar um caractere \, é usado \\e para a aspas simples o \' e aspas duplas \". Também é permitido especificar uma caráctere por meio de seu código ASCII, usado \seguido por exatamente três dígitos. Exemplos de literais caractere: 'a', '\n', '\t', '\\', '\065';
- Um **literal lógico** é um dos valores **true** que denota o valor booleano verdadeiro ou **false** que denota o valor booleano falso;

[1] http://www.asciitable.com

- O **literal nulo** é **null**;
- Os símbolos usados para **operadores** e **separadores** são *(*, **)**, **[**, **]**, **{**, **}**, >, **;**, **:**, **::**, **.**, **,**, =, <, ==, **!=**, +, **-**, *****, /, **%**, **&&**, **&**, e **!**.
- **Palavras reservadas** são identificadores especiais usados pela linguagem como **if**, **else**, **iterate**, **data**, **class**, **instance**, **for**, **return**, **new**, **Int**, **Char**, **Float**, **Bool** e **Void**, etc...

Todos os nomes de tipos, comandos e literais são palavras reservadas pela linguagem. Há dois tipos de comentários: comentário de uma linha e de múltiplas linhas. O comentário de uma linha começa com `--` e se estende até a quebra de linha. O comentário de múltiplas linhas começa com {- e se estende até os caracteres de fechamento do comentário, -}. A linguagem não suporta comentários aninhados.

3 Semântica de Lang

A semântica de um programa $lang^2$ é definida em termos de um estado formado por uma memória local M, uma memória global H (ambas associam identificadores a valores) e uma pilha P de valores, que será usada para computações temporárias. Escreveremos $v : P$ para indicar que v foi empilhado no topo da pilha. A pilha vazia será denotada por $\square$. Denotaremos a associação de um identificador I a um valor V na memória como $M[I = V]$. Caso o identificador já esteja associado a algum valor na memória, este será substituído pelo novo valor. A notação $M(I)$ será usada para denotar o valor V associado a um identificador I. Se não houver valor associado ao identificador, a operação resultará em erro. A notação $M/\{I\}$ denota a memória obtida a partir da subtração da associação de $I = V$ de M. Remoções de múltiplas associações simultâneas serão denotadas por $M/\{I_1, \ldots, I_n\}$. Adicionalmente, vamos supor a existência de um contexto de funções $\mathbb{F}$ no qual estão armazenadas todas as funções definidas pelo programa. Também será assumido que $\mathbb{F}$ é um contexto imutável durante a execução do programa.

Para simplificar a escrita, usaremos $M[I_1 = V_1, I_2 = V_2, \ldots, I_n = V_n]$ para especificar várias atribuições simultaneamente na memória. A memória vazia será denotada por $[]$. A mesma notação será usada para a memória global, compartilhada entre funções. Por fim, usaremos a notação α para denotar identificadores únicos utilizados na memória global.

Escreveremos $\langle H, M, P\rangle$ para denotar o estado. Os identificadores serão representados por strings de caracteres. Os valores podem ser inteiros, valores de ponto flutuante, booleanos, a constante *null*, vetores ou registros.

O conjunto de valores $\mathcal{L}$ da linguagem $lang^2$ pode ser descrito recursivamente como:

1. Se $i \in \mathbb{Z}$, então $i \in \mathcal{L}$.
2. Se $f \in \mathbb{R}$, então $f \in \mathcal{L}$.
3. Se $b \in \{True, False\}$, então $b \in \mathcal{L}$.
4. Se c é um caractere ASCII, então $c \in \mathcal{L}$.
5. O valor $null \in \mathcal{L}$.

6. Sejam n valores $v_0, \ldots, v_{n-1}$ e seja A um arranjo de n posições tal que, para $0 \leq i < n$, $A[i] = v_i$. Então, $A \in \mathcal{L}$.

7. Seja uma sequência I de n identificadores e uma sequência V de n valores. O mapeamento $\{(I_i \rightarrow V_i)\}$, $0 \leq i < n$, pertence a $\mathcal{L}$. Valores do tipo mapeamento podem ter seus campos acessados com o operador ".". Por exemplo, se $V = \{x = 0, y = 1\}$, então $V.x$ denota o valor 0. Além disso, os campos podem ter seus valores sobrescritos.

8. α: o identificador (ou endereço) em uma memória global será considerado como um valor.

A regra número 7 descreve valores de tipo registro. Note que registros são associações entre identificadores e valores. Essa associação é única no escopo do registro. Contudo, um campo de um registro pode ser outro registro que possua os mesmos nomes de campo. Um exemplo dessa situação é um tipo de dados recursivo, como uma lista.

A regra 6 descreve vetores como uma sequência indexada de valores arbitrários.

Dadas as definições de valores e de estado, podemos descrever a semântica de cada comando da linguagem $lang^2$. A semântica apresentada a seguir é informal, ou seja, não descreve de forma precisa o comportamento de um programa. As lacunas nas definições devem ser preenchidas levando-se em conta parcimônia e clareza do comportamento do programa.

Expressões A semântica de expressões é dada pela função $S : e \rightarrow \langle H, M, P\rangle \rightarrow \langle H', M', P'\rangle$, definida pelos casos a seguir.

- $S(\text{INT}, \langle H, M, P\rangle) = \langle H, M, i : P\rangle$, onde i é o inteiro denotado por INT.
- $S(\text{FLOAT}, \langle H, M, P\rangle) = \langle H, M, f : P\rangle$, onde f é o valor de ponto flutuante denotado por $FLOAT$.
- $S(\textbf{true}, \langle H, M, P\rangle) = \langle H, M, true : P\rangle$.
- $S(\textbf{false}, \langle H, M, P\rangle) = \langle H, M, false : P\rangle$.
- $S(\textbf{null}, \langle H, M, P\rangle) = \langle H, M, null : P\rangle$.
- $S(e_1 \oplus e_2, \langle H, M, P\rangle) = \langle H, M, f_{op}(\oplus, v_1, v_2) : P\rangle$, onde $\oplus$ é um operador binário qualquer, $\langle H, M, v_1 : P\rangle = S(e_1, \langle H, M, P\rangle)$, $\langle H, M, v_2 : v_1 : P\rangle = S(e_2, \langle H, M, v_1 : P\rangle)$ e a função f_{op} mapeia cada operador para sua semântica correspondente na álgebra. Por exemplo, $f_{op}(+, 1, 2) = 3$. O operador f_{op} está definido para operadores aritméticos, booleanos e relacionais, bem como para valores inteiros, booleanos e de ponto flutuante. No entanto, ele não é definido para vetores e registros.
- $S(lvalue[e], \langle H, M, P\rangle) = \langle H, M, A[i] : P\rangle$, onde $\langle H, M, A : P\rangle = S(lvalue, \langle H, M, P\rangle)$, A é um arranjo e $\langle H, M, i : A : P\rangle = S(e, \langle H, M, A : P\rangle)$.
- $S(lvalue.ID, \langle H, M, P\rangle) = \langle H, M, V : P\rangle$, onde $\langle H, M, R : P\rangle = S(lvalue, \langle H, M, P\rangle)$, R é um mapeamento entre identificadores e valores, e $(ID, V) \in R$.
- $S(ID, \langle H, M, P\rangle) = \langle H, M, M(ID) : P\rangle$.
- $S(ID(e_1 \ldots e_n)[e], \langle H, M, P\rangle) = \langle H, M, V_k : P\rangle$, onde:

- Verifica-se que há apenas um ID tal que $ID\ ID_1 \ldots ID_n :: \tau\ C \in \mathbb{F}$.
- Avaliam-se as expressões dos argumentos: $\langle H, M, V_1 : \ldots : V_n : P\rangle = S(e_1, S(e_2, \ldots, S(e_n, \langle H, M, P\rangle)$
- Constrói-se uma nova memória $M'[ID_1 = V_1, \ldots, ID_n = V_n]$.
- Executa-se $S(C, \langle H, M', P\rangle) = \langle H, M'', V_1 : \ldots : V_m\rangle$, sendo C o corpo da função executada.
- Avalia-se a expressão que denota o valor de retorno a ser usado: $S(e, \langle H, M, V_1 : \ldots : V_m\rangle) = \langle H, M, k : V_1 : \ldots : V_m\rangle$, em que $0 \leq k \leq m$.
- Finalmente, salva-se apenas o k-ésimo valor de retorno na pilha, removendo-se os demais valores: $\langle H, M, V_k : P\rangle$.

- $S(new\ \ TYPE\ \ n, \langle H, M, P\rangle) = \langle H[\alpha = V], M, \alpha : P\rangle$, onde o valor V é obtido por uma instanciação do tipo $TYPE$, e o valor n pode ser omitido caso $TYPE$ seja o nome de um registro. Note que a operação *new* insere, na memória global, o valor associado a um identificador α e insere na pilha o nome, ou endereço, do valor na memória global.

Instanciação de valores Uma instanciação de valores $new\ \ TYPE\ \ n$ ocorre da seguinte forma:

- Se o tipo a ser instanciado for primitivo (*Int*, *Float*, *Char* ou *Bool*), deve-se criar um vetor de valores com tamanho $n \in \mathbb{Z}$, $n \geq 0$.
- Se o tipo a ser instanciado for um tipo definido pelo usuário e o tamanho for omitido, deve-se utilizar a definição do tipo de dados para criar um mapeamento dos identificadores de cada campo para seus valores padrão correspondentes. O valor padrão de tipos numéricos é 0, o valor padrão de um tipo *Char* é '\000' e o valor padrão de um tipo *Bool* é $false$. O valor padrão de um tipo definido pelo usuário é *null*. Caso o valor n seja especificado, deve-se criar um vetor com n elementos preenchidos com o valor *null*.

Semântica de comandos A semântica de comandos é dada pela função $C : c \rightarrow \langle H, M, P\rangle \rightarrow \langle H', M', P'\rangle$, definida pelos casos a seguir:

1. $C(lvalue = exp, \langle H, M, P\rangle) =$ Seja $\langle H', M, V : P\rangle = S(exp, \langle H, M, P\rangle)$. Caso V não seja um identificador de memória global, o resultado é $update(\langle H', M, P\rangle, lvalue, V)$, onde *update* é uma operação que escreve o valor no local apropriado da memória, retornando um novo estado. Note que essa função pode precisar acessar a memória global para atualizar um campo de registro ou uma posição de um vetor. Caso $V = \alpha$, então $update(\langle H', M, P\rangle, lvalue, H'[V])$.
2. $C(if\ exp\ stmtBlock, \langle H, M, P\rangle) =$ Seja $\langle H', M, V : P\rangle = S(exp, \langle H, M, P\rangle)$. Caso $V = true$, então $C(stmtBlock, \langle H', M, P\rangle)$; caso contrário, $\langle H', M, P\rangle$.
3. $C(if\ exp\ stmtBlock_1\ stmtBlock_2, \langle H, M, P\rangle) =$ Seja $\langle H', M, V : P\rangle = S(exp, \langle H, M, P\rangle)$. Caso $V = true$, então $C(stmtBlock_1, \langle H', M, P\rangle)$; caso contrário, $C(stmtBlock_2, \langle H', M, P\rangle)$.
4. $C(iterate\ exp\ stmtBlock, \langle H, M, P\rangle)$: Seja $\langle H', M', V : P\rangle = S(exp, \langle H, M, P\rangle)$, onde $M' = M[I_c = V]$ é uma nova memória que mapeia um identificador I_c, que não ocorre em lugar algum do programa[2], para o valor V.

[2]Tais identificadores são referidos como variáveis *fresh*.

(a) Caso $M[I_c] = 0$, então o resultado é $\langle H', M'/\{I_c\}, P\rangle$, concluindo a execução do *iterate*.

(b) Caso $M[I_c] = i$, com $i > 0$, então $\langle H'', M'', P'\rangle = C(stmtBlock, \langle H', M', P\rangle)$ e, em seguida, $M''' = M''[I_c = M''[I_c] - 1]$, e o estado resultante é $\langle H'', M''', P'\rangle$. Considerando a memória desse novo estado, retomamos a execução a partir da regra 4a.

(c) Caso $V = A$, sendo A um arranjo de tamanho n, considere n como um valor inteiro e prossiga de acordo com as regras 4a e 4b.

5. $C(iterate\ id : exp\ stmtBlock, \langle H, M, P\rangle)$: Nesse caso, considera-se id como uma variável de acesso ao contador. Além disso, a variável id, se declarada no comando *iterate*, é local ao bloco do comando, isto é, não deve ser acessível fora dele. Caso a variável já tenha sido previamente declarada, ela permanecerá acessível fora do bloco do comando. Seja $\langle H', M', V : P\rangle = S(exp, \langle H, M, P\rangle)$, onde $M' = M[I_c = V, id = V]$ é uma nova memória que mapeia um identificador *fresh* I_c.

   (a) Caso $M[I_c] = 0$, então o resultado é $\langle H', M'/\{I_c\}, P\rangle$, se id foi declarada previamente ao comando *iterate*, ou $\langle H', M'/\{I_c, id\}, P\rangle$, se id foi declarada no comando *iterate*. Esse passo conclui a execução do *iterate*.

   (b) Caso $M[I_c] = i$, com $i > 0$, então $\langle H'', M'', P'\rangle = C(stmtBlock, \langle H', M', P\rangle)$ e, em seguida, $M''' = M''[I_c = M''[I_c] - 1, id = M''[I_c] - 1]$, e o estado resultante é $\langle H'', M''', P'\rangle$. Considerando a memória desse novo estado, retomamos a execução a partir da regra 5a.

   (c) Caso $V = A$, sendo A um arranjo de tamanho n, considere $M' = M[I_c = n, id = A[n - M[I_c]]]$ e:

      i. Caso $M[I_c] = 0$, então o resultado é $\langle H', M'/\{I_c\}, P\rangle$, se id foi declarada previamente ao comando *iterate*, ou $\langle H', M'/\{I_c, id\}, P\rangle$, se id foi declarada no comando *iterate*. Esse passo conclui a execução do *iterate*.

      ii. Caso $M[I_c] = i$, com $i > 0$, então $\langle H'', M'', P'\rangle = C(stmtBlock, \langle H', M', P\rangle)$ e, em seguida, $M''' = M''[I_c = M''[I_c] - 1, id = A[M''[I_c] - 1]]$, e o estado resultante é $\langle H'', M''', P'\rangle$. Considerando a memória desse novo estado, retomamos a execução a partir da regra 5(c)i.

6. $C(return\ exp_1 \ldots exp_n, \langle H, M, P\rangle)$:

   - $\langle H_1, M, V_1 : P\rangle = S(exp_1, \langle H, M, P\rangle)$
   - $\ldots$
   - $\langle H_n, M, V_n \ldots V_1 : P\rangle = S(exp_n, \langle H_{n-1}, M, V_{n-1} \ldots V_1 : P\rangle)$

   O estado resultante é $\langle H_n, M, V_n \ldots V_1 : P\rangle$. Adicionalmente, o comando *return* deve causar o término da execução da função corrente. Após a chamada de *return*, nenhum comando subsequente na função deve ser executado, e o fluxo de controle deve retornar para quem chamou a função.

7. A semântica da chamada de função no nível de comando é análoga à chamada de função em expressões, exceto pelo fato de que os valores de retorno sempre devem ser removidos da pilha e associados aos *lvalues* anotados entre $<$ e $>$, respectivamente.

8. A semântica de uma sequência $C(\{c_1\, c_2 \ldots c_n\}, \langle H, M, P\rangle) = \langle H'', M'', P''\rangle$, onde $\langle H', M', P'\rangle = C(c_1, \langle H, M, P\rangle)$ e $\langle H'', M'', P''\rangle = C(\{c_2 \ldots c_n\}, \langle H', M', P'\rangle)$.

Além da parte formal apresentada neste texto, considere também que programas corretos em $lang^2$ sempre satisfazem as seguintes afirmações:

- Os operadores aritméticos são homogêneos em relação aos tipos de seus operandos. Ou seja, um operador aritmético $a \oplus b$ pode operar apenas sobre argumentos inteiros (ambos a e b) ou sobre argumentos de ponto flutuante, mas nunca sobre um argumento inteiro e um argumento de ponto flutuante.
- Todas as funções devem ser declaradas antes de seu uso, e funções recursivas e mutuamente recursivas são suportadas pela linguagem.
- Não deve existir código morto (sequência de comandos que nunca é alcançada pelo fluxo de controle).
- As chamadas de funções são sempre realizadas observando-se o **número correto de argumentos** esperados na definição.
- A quantidade de expressões passadas ao comando Códigoreturn sempre corresponde à quantidade de termos na anotação de tipo da função.
- Toda variável deve ser declarada antes de ser usada.
- Todo tipo de dado definido pelo usuário deve ser declarado antes de ser usado, sendo permitido o uso de tipos de dados recursivos e mutuamente recursivos.

3.1 Funções Primitivas da Linguagem $lang^2$

A linguagem $lang^2$ não define construções sintáticas para entrada e saída de dados, nem para algumas outras funcionalidades. Em vez disso, define algumas funções como primitivas.

Uma função primitiva é uma função que existe na linguagem, mas não é necessariamente definida na própria linguagem. Tal mecanismo permite expressar funcionalidades como entrada e saída (I/O), que não são triviais de serem definidas na linguagem. Outro uso comum é a integração de recursos de baixo nível que podem não ser possíveis de se expressar em alto nível, mas que podem aparecer na linguagem como funções primitivas.

Neste momento, as seguintes funções primitivas devem ser consideradas como parte da linguagem $lang^2$:

- `print :: Char -> Void`. Imprime um único caractere na saída padrão.
- `printb :: Char[] -> Int -> Int -> Void`. Imprime um intervalo de um vetor de caracteres na saída padrão. Por exemplo, `printb(buff, i, l)` imprime `l` caracteres, começando da posição `i` do vetor.
- `read :: Char`. Lê um único caractere da entrada padrão.
- `readb :: Char[] -> Int -> Int -> Int`. Lê uma sequência de caracteres da entrada, transferindo-os para um vetor de caracteres. Retorna o número de caracteres efetivamente lidos. Por exemplo, `readb(buff, i, l)` lê até `l` caracteres da entrada e os armazena em `buff` a partir da posição `i`. Caso a quantidade de dados a serem lidos exceda o limite do vetor (a partir da posição `i`), a leitura é encerrada.

4 Semântica Estática de $lang^2$

A semântica estática de $lang^2$ é descrita como um conjunto de julgamentos que determinam se um certo programa $lang^2$ é considerado válido ou não. Para isso, descreveremos a estrutura da semântica estática de acordo com os níveis de sintaxe de $lang^2$. A seção 4.1 descreve a sintaxe abstrata e os contextos utilizados para definir todas as regras semânticas. A seções 4.2, 4.3 e 4.4 descrevem regras para expressões, comandos e declarações, respectivamente. A seção 4.5 descreve regras semânticas para programas.

4.1 Sintaxe abstrata e contextos

A sintaxe abstrata de expressões é definida pela seguinte gramática, em que $\bullet$ denota uma lista vazia, n denota constantes inteiras, f constantes de ponto flutuante, c caracteres, $\circ$ denota operadores binários, $\surd$ denota operadores unários, v denota um identificador, a denota uma variável de tipo.

$$
\begin{array}{lcl}
e & \rightarrow & n \mid f \mid c \mid \textbf{true} \mid \textbf{false} \mid \textbf{null} \mid e \circ e \mid \surd e \mid e[e] \mid v(es)[es] \mid \textbf{new}\ \rho\ es\ lv \\
lv & \rightarrow & v \mid lv\ .\ v \mid lv[e] \\
\circ & \rightarrow & + \mid - \mid * \mid / \mid \% \mid \&\& \mid == \mid != \mid < \\
\surd & \rightarrow & ! \mid - \\
es & \rightarrow & e\ es \mid \bullet
\end{array}
$$

Expressões são formadas por literais, constantes booleanas, o valor **null**, operadores binários, unários, acesso a arranjos, chamadas de funções, alocação de memória, variáveis e acesso a campos de registros.

A seguir, apresentamos a sintaxe abstrata de comandos.

$$
\begin{array}{lcl}
cdm & \rightarrow & lv = e \\
 & \mid & \textbf{if}\ e\ \textbf{then}\ cs\ \textbf{else}\ cs \\
 & \mid & \textbf{iterate}\ e\ cs \\
 & \mid & \textbf{return}\ es \\
 & \mid & v(es)\ lvs \\
cmds & \rightarrow & cmd\ cmds \mid \bullet \\
lvs & \rightarrow & lv\ lvs \mid \bullet
\end{array}
$$

Comandos podem ser atribuições, declaração de variáveis, condicionais, repetição, retorno de função e chamada de função.

Tipos podem ser tipos básicos, tipos definidos pelo usuário, arranjos, tipos Restritos ou tuplas de tipos.

$$
\begin{array}{lcl}
\tau & \rightarrow & \rho \mid \tau \mapsto \tau \mid \tau\,[\,] \mid v\ a => \tau \\
\rho & \rightarrow & \textbf{Int} \mid \textbf{Float} \mid \textbf{Char} \mid \textbf{Bool} \mid \textbf{Void} \mid v \mid a \mid (\rho_1, ..., \rho_n)
\end{array}
$$

Programas *lang* são formados por declarações de tipos, funções, classes de tipos ou instâncias

cuja sintaxe abstrata é apresentada a seguir.

$$
\begin{array}{lcl}
prog & \rightarrow & def\ \ prog \mid \bullet \\
def & \rightarrow & nd \mid fun \mid cls \mid ist \\
nd & \rightarrow & v\,\{fds\} \\
fun & \rightarrow & v\,vs :: ts\ \ cmds \\
fd & \rightarrow & v{::}\tau \\
fds & \rightarrow & fd\ \ fds \mid \bullet \\
ts & \rightarrow & \tau\ \ ts \mid \bullet \\
vs & \rightarrow & v\,vs \mid \\
cls & \rightarrow & \textbf{class}\ v\,ps \\
ps & \rightarrow & v\ ::\ ts\,ps \mid \bullet \\
ist & \rightarrow & \textbf{instance}\ v\,\rho\,fs \\
fs & \rightarrow & fun\,fs \mid \bullet
\end{array}
$$

Para verificação da semântica estática de programas *lang* utilizaremos três contextos:

- Θ: contexto contendo os tipos de funções definidas pelo usuário. Este contexto é um conjunto de pares $(f, [v_1 : \tau_1, ..., v_n : \tau_n] \rightarrow [\tau'_1, ..., \tau'_m])$, em que f é o nome da função e $[v_1 : \tau_1, ..., v_n : \tau_n] \rightarrow [\tau'_1, ..., \tau'_m]$ o seu tipo. Note que o tipo dos argumentos é representado por uma sequência de pares de identificadores e seus tipos e o tipo de
- Γ: contexto contendo definições de variáveis e operadores de *lang*. Este contexto é formado por pares (x, τ) em que x é um identificador e τ o seu respectivo tipo. N retorno como uma lista de tipos.
- Δ: contexto contendo informações sobre os campos de registros definidos pelo usuário. O contexto Δ é um conjunto de pares da forma (x, S) em que x é o nome do tipo e S é um conjunto de formado por pares (v, τ) em que $v :: \tau$ é um campo do tipo x.este contexto armazemos os tipos de todos os operadores binários e unários de *lang*.
- I Contexto de instâncias defindias. Contém uma lista de todas as instâncias de tipos definidas no programa.Uma instância será denotada pelo termo $v\,\rho$ no qual v é o nome da classe e ρ um tipo simples, excluindo-se variáveis de tipos e o tipo **Void**.

A notação $\Gamma(x) = \tau$ denota que $(x, \tau) \in \Gamma$ e $\Gamma, x : \tau$ denota $\Gamma \cup \{(x, \tau)\}$. Estas operações aplicam-se também aos contextos Δ e Θ. Finalmente, representamos um contexto vazio por $\bullet$.

4.2 Semântica estática de expressões

A semântica estática de expressões Lang são definidas como um julgamento de forma $\Theta; \Delta; \Gamma \vdash_e e : \tau$ que representa que a expressão e possui o tipo τ sobre os contextos Θ, Δ e Γ.

Iniciaremos a descrição com regras para constantes, que possuem regras para de tipos imediatas: por exemplo, a primeira regra mostra que constantes inteiras (n) possuem o tipo **Int**. A única pecurialidade é que a constante **null** pode possuir qualquer tipo de arranjo ou de registro, mas não tipos primitivos da linguagem.

Variáveis possuem o tipo que é atribuído a elas pelo contexto de tipos Γ.

$$\frac{}{I;\Theta;\Delta;\Gamma \vdash_e n : \textbf{Int}} \qquad \frac{}{I;\Theta;\Delta;\Gamma \vdash_e f : \textbf{Float}} \qquad \frac{}{I;\Theta;\Delta;\Gamma \vdash_e c : \textbf{Char}}$$

$$\frac{}{I;\Theta;\Delta;\Gamma \vdash_e \textbf{true} : \textbf{Bool}} \quad \frac{}{I;\Theta;\Delta;\Gamma \vdash_e \textbf{false} : \textbf{Bool}} \quad \frac{\tau \notin \{\textbf{Int}, \textbf{Char}, \textbf{Float}, \textbf{Bool}\}}{I;\Theta;\Delta;\Gamma \vdash_e \textbf{null} : \tau}$$

$$\frac{\Gamma(x) = \tau}{I;\Theta;\Delta;\Gamma \vdash_e x : \tau}$$

A verificação de acesso a campos em valores de registros é verificada da seguinte forma:

1. Primeiro, obtemos o tipo da variável x_1, $I;\Theta;\Delta;\Gamma \vdash_e x_1 : \tau_1$;
2. Em seguida, obtemos o conjunto de campos de τ_1, $\Delta(\tau_1) = S_1$;
3. Finalmente, obtemos o tipo de x_2 no conjunto de campos de τ_1, S_1.

$$\frac{I;\Theta;\Delta;\Gamma \vdash_e x_1 : \tau_1 \quad \Delta(\tau_1) = S_1 \quad (x_2, \tau) \in S_1}{I;\Theta;\Delta;\Gamma \vdash_e x_1.x_2 : \tau}$$

A verificação de operadores binários e unários são como se segue:

$$\frac{I;\Theta;\Delta;\Gamma \vdash_e e_1 : \tau_1 \quad I;\Theta;\Delta;\Gamma \vdash_e e_2 : \tau_2 \quad I;\Theta;\Delta;\Gamma(\circ) = \tau_1 \mapsto \tau_2 \mapsto \tau}{I;\Theta;\Delta;\Gamma \vdash_e e_1 \circ e_2 : \tau}$$

$$\frac{I;\Theta;\Delta;\Gamma \vdash_e e_1 : \tau_1 \quad I;\Theta;\Delta;\Gamma(\surd) = \tau_1 \mapsto \tau}{I;\Theta;\Delta;\Gamma \vdash_e \surd\, e_1 : \tau}$$

Uma expressão $e_1 \circ e_2$ possui o tipo τ se:

1. O operador $\circ$ possui o tipo $\tau_1 \mapsto \tau_2 \mapsto \tau$;
2. a expressão e_1 possui tipo τ_1, isto é, $\Theta;\Delta;\Gamma \vdash_e e_1 : \tau_1$;
3. a expressão e_2 possui o tipo τ_2, isto é, $\Theta;\Delta;\Gamma \vdash_e e_2 : \tau_2$.

A verificação dos operadores unários é similar.

Evidentemente, que o tipo dos operadores deve estar presente no contexto Γ. A tabela a seguir, descreve o tipo de cada um dos operadores disponível em *lang*.

Operador	Tipo
$+, -, *, /,$	$a \mapsto a \mapsto a$, em que $a \in \{\mathbf{Int}, \mathbf{Float}\}$
$\%$	$\mathbf{Int} \mapsto \mathbf{Int} \mapsto \mathbf{Int}$
$==, !=, <$	$a \mapsto a \mapsto \mathbf{Bool}$, em que $a \in \{\mathbf{Int}, \mathbf{Float}, \mathbf{Char}\}$
&&	$\mathbf{Bool} \mapsto \mathbf{Bool} \mapsto \mathbf{Bool}$
!	$\mathbf{Bool} \mapsto \mathbf{Bool}$
-	$a \mapsto a$, em que $a \in \{\mathbf{Int}, \mathbf{Float}\}$
print	$\mathbf{Char} \mapsto \mathbf{Void}$
printb	$\mathbf{Char[]} \mapsto \mathbf{Int} \mapsto \mathbf{Int} \mapsto \mathbf{Void}$
read	$Void \mapsto \mathbf{Char}$
read	$\mathbf{Char[]} \mapsto \mathbf{Int} \mapsto \mathbf{Int} \mapsto \mathbf{Void}$

O contexto inicial formado pelas definições da tabela anterior será chamado de Θ_0.

Além de tipos básicos, lang permite a definição de arranjos. A verificação de accesso a arranjos é feita conforme a seguinte regra:

$$\frac{I;\Theta;\Delta;\Gamma \vdash_e e_1 : \tau\,[\,] \quad I;\Theta;\Delta;\Gamma \vdash_e e_2 : \mathbf{Int}}{I;\Theta;\Delta;\Gamma \vdash_e e_1[e_2] : \tau}$$

A regra de verificação de chamadas de funções é feita da seguinte forma:

1. Primeiro obtemos o tipo da função utilizando o contexto Θ e seu identificador: $\Theta(v) = \tau_f$.
2. Na sequência verificamos os tipos de cada argumento da chamada de função com o seu respectivo tipo: $I;\Theta;\Delta;\Gamma \vdash_e e_i : \tau_i$, em que m é o número de argumentos da função, $1 \leq i \leq m$.
3. A partir da chama de funcão, construímos o tipo τ_i.
4. Caso $\tau_f = v\,a => \tau$, verificamos se existe algum tipo ρ tal que se sustituímos a por ρ em τ_f obtemos τ_i. Em caso afirmativo consultamos I para saber se existe a instância $v\,\rho$.
5. Verificar que os indicadores de acesso a valores de retorno estão dentro do limite do número de valores de retorno, p.

$$\frac{\Theta(v) = \tau_1 \mapsto ... \mapsto \tau_m \mapsto (\tau_1, ..., \tau_j) \quad I;\Theta;\Delta;\Gamma \vdash_e e_i : \tau_i \quad I;\Theta;\Delta;\Gamma \vdash_e n : \mathbf{Int} \quad \begin{array}{c} 1 \leq n \leq j \\ 1 \leq i \leq m \end{array}}{I;\Theta;\Delta;\Gamma \vdash_e v(e_1, ..., e_m)[n] : \tau_n}$$

$$\frac{\begin{array}{ccc} I;\Theta;\Delta;\Gamma \vdash_e n : \mathbf{Int} & & 1 \leq n \leq j \\ I;\Theta;\Delta;\Gamma \vdash_e e_i : \tau_{ci} & \tau_{ir} = \tau_{c0} \mapsto ... \mapsto \tau_{cm} \mapsto (\tau_1, ..., \tau_j) & \exists \rho.v\,\rho \in I \\ \Theta(v) = v\,a{=>}\tau_1 \mapsto ... \mapsto \tau_m \mapsto (\tau_1, ..., \tau_j) & \tau_{ir} = \mathbf{subs}(\rho, \tau_1 \mapsto ... \mapsto \tau_m \mapsto (\tau_1, ..., \tau_j)) & 1 \leq ci \leq m \end{array}}{I;\Theta;\Delta;\Gamma \vdash_e v(e_1, ..., e_m)[n] : \tau_n}$$

A expressão para alocação dinâmica pode ser utilizada sobre tipos de arranjos ou tipos de registros. A regra seguinte mostra como verificar a alocação de arranjos, em que $\mathbf{dom}(\Delta)$ representa o domínio do contexto Δ, isto é $\mathbf{dom}(\Delta) = \{x \mid \exists S.\Delta(x) = S\}$.

$$\frac{\forall e.e \in es \rightarrow \Theta;\Delta;\Gamma \vdash_e e : \mathbf{Int} \quad \tau \notin \{\mathbf{Int}, \mathbf{Float}, \mathbf{Char}, \mathbf{Bool}\} \cup \mathbf{dom}(\Delta)}{I;\Theta;\Delta;\Gamma \vdash_e \mathbf{new}\ \tau\ es : \tau}$$

A regra funciona da seguinte forma:

1. Todas as expressões $e \in es$ devem ter tipo **Int**, para definição de limites de arranjos.
2. O tipo τ deve ser um tipo de arranjo, isto é, não deve ser um tipo primitivo nem um tipo definido pelo usuário.

A próxima regra mostra como verificar a alocação de tipos definidos pelo usuário.

$$\frac{\tau \in \mathbf{dom}(\Delta)}{I : \Theta;\Delta;\Gamma \vdash_e \mathbf{new}\ \tau\ : \tau}$$

Note que a única restrição é que o tipo a ser alocado faz parte do domínio do ambiente Δ.

4.3 Semântica estática de comandos

A semântica estática de comandos é dada por um julgamento $I;\Theta;\Delta;\Gamma;V \vdash_c\ cmd \rightsquigarrow V';\Gamma'$ que denota que o comando c é bem formado nos contextos Θ, Δ e Γ. Note que o comando c pode modificar o contexto Γ por incluir uma nova variável. Por isso, este julgamento produz, como resultado, um novo contexto Γ' e conjuntos de variáveis incluídas V e V'. Usaremos este conjunto para controlar o escopo de visibilidade de identificadores. Adicionalmente, o julgamento $I\Theta;\Delta;\Gamma;V \vdash_{cs}\ cmd \rightsquigarrow V' \times \Gamma'$ lida com a verificação de blocos.

A primeira regra para blocos lida com a marcação de final de um bloco, $\bullet$. Esta regra simplesmente remove do contexto de tipos de resultado, Γ', todos os identificadores que foram introduzidos no bloco atual e que estão armazenados no conjunto de variáveis V.

$$\frac{\Gamma' = \{(x,\tau) \mid x \notin V \wedge \Gamma(x) = \tau\}}{I;\Theta;\Delta;\Gamma;V \vdash_{cs} \bullet \rightsquigarrow \emptyset;\Gamma'}$$

A próxima regra lida com blocos não vazios e basicamente é responsável por processar o primeiro comando do bloco e repassar os resultados deste processamento para o restante dos comandos deste bloco.

$$\frac{I;\Theta;\Delta;\Gamma;V \vdash_c cmd \rightsquigarrow V_1;\Gamma_1 \quad I;\Theta;\Delta;\Gamma_1;V_1 \vdash_{cs} cmds \rightsquigarrow V';\Gamma'}{I;\Theta;\Delta;\Gamma;V \vdash_{cs} cmd\ \ cmds \rightsquigarrow V';\Gamma'}$$

A primeira regra para comandos lida com a atribuição. Esta regra é formada pelos seguintes passos:

1. Verificamos o tipo do lado esquerdo da atribuição;

2. Verificamos o tipo do lado direito da atribuição;

$$\frac{I;\Theta;\Delta;\Gamma \vdash_e lvs : \tau \quad I;\Theta;\Delta;\Gamma \vdash_e e : \tau}{I;\Theta;\Delta;\Gamma;V \vdash_c lvs \ = \ e : V;\Gamma}$$

É importante notar que esta regra não modifica o conjunto de variáveis introduzidas e nem o contexto de tipos, pois não há inclusão de novas variáveis no contexto.

A verificação de declaração de variáveis locais é feita de acordo com os seguintes passos:

1. Iniciamos verificando que a expressão de inicialização da variável v possui o mesmo tipo τ de sua declaração.
2. A regra retorna o contexto modificado contendo o tipo da nova variável e a inclui no conjunto de variáveis definidas no bloco atual.

$$\frac{I;\Theta;\Delta;\Gamma \vdash_e e : \tau \quad \nexists \tau.(v,\tau) \in \Gamma \quad V' = \{v\} \cup V \quad \Gamma' = \Gamma, v : \tau}{I;\Theta;\Delta;\Gamma;V \vdash_c \ v \ = \ e \rightsquigarrow V';\Gamma'}$$

Na sequência, apresentamos a regra para validação de comandos condicionais. Primeiro, verificamos que o tipo da expressão e deve ser **Bool** e, na sequência, validamos os blocos de comandos cs_1 e cs_2 ignorando as eventuais declarações introduzidas nestes blocos.

$$\frac{I;\Theta;\Delta;\Gamma \vdash_e e : \mathbf{Bool} \quad I;\Theta;\Delta;\Gamma;V \vdash_c cs_1 \rightsquigarrow V_1;\Gamma_1 \quad I;\Theta;\Delta;\Gamma;V \vdash_c cs_2 \rightsquigarrow V_2;\Gamma_2}{I;\Theta;\Delta;\Gamma;V \vdash_c \mathbf{if}\ e\ \mathbf{then}\ cs_1\ \mathbf{else}\ cs_2 \rightsquigarrow V;\Gamma}$$

A seguir, apresentamos a regra para verificar comandos de repetição. Iniciamos a verificação por demandar que a expressão e possua tipo **Int** e na sequência verificamos o bloco de comandos do comando **iterate**, descartando modificações no contexto de tipos. Lembrado que e pode ter tipo inteiro ou um tipo arranjo e o comando iterate pode ou não possuir uma variável.

$$\frac{I;\Theta;\Delta;\Gamma \vdash_e e : \mathbf{Int} \quad I;\Theta;\Delta;\Gamma;V \vdash_c cs \rightsquigarrow V';\Gamma'}{I;\Theta;\Delta;\Gamma;V \vdash_c \mathbf{iterate}\, e\, cs \rightsquigarrow V;\Gamma}$$

$$\frac{I;\Theta;\Delta;\Gamma \vdash_e e : \mathbf{Int} \quad I;\Theta;\Delta;\Gamma;V' \vdash_c cs \rightsquigarrow V'';\Gamma' \quad \begin{array}{r}\Gamma' = \Gamma, (v, Int) \text{ se } \nexists \tau.(c,\tau) \in \Gamma \text{ ou } \Gamma \text{ caso contrário} \\ V' = V \cup v, \text{ se } \nexists \tau.(c,\tau) \in \Gamma \text{ ou } V \text{ caso contrário}\end{array}}{I;\Theta;\Delta;\Gamma;V \vdash_c \mathbf{iterate}\, e\, v\, cs \rightsquigarrow V;\Gamma}$$

$$\frac{I;\Theta;\Delta;\Gamma \vdash_e e : \tau[] \quad I;\Theta;\Delta;\Gamma;V \vdash_c cs \rightsquigarrow V';\Gamma'}{I;\Theta;\Delta;\Gamma;V \vdash_c \mathbf{iterate}\, e\, cs \rightsquigarrow V;\Gamma}$$

$$\frac{I;\Theta;\Delta;\Gamma \vdash_e e : \tau_1[] \quad I;\Theta;\Delta;\Gamma;V' \vdash_c cs \rightsquigarrow V'';\Gamma' \quad \begin{array}{r}\Gamma' = \Gamma, (v, \tau_1) \text{ se } \nexists \tau.(c,\tau) \in \Gamma \text{ ou } \Gamma \text{ caso contrário} \\ V' = V \cup v, \text{ se } \nexists \tau.(c,\tau) \in \Gamma \text{ ou } V \text{ caso contrário}\end{array}}{I;\Theta;\Delta;\Gamma;V \vdash_c \mathbf{iterate}\, e\, v\, cs \rightsquigarrow V;\Gamma}$$

Para verificarmos o comando **return**, precisamos determinar se a lista de valores retornados possui o mesmo tipo que o retorno anotado no cabeçalho da função. Denotamos por $\Theta_\tau = [\tau_1, ..., \tau_m]$ a tupla de tipos do retorno da função. A regra seguinte mostra como validar o comando **return**.

$$\frac{I; \Theta_\tau = (\tau_1, ..., \tau_m) \quad I; \Theta; \Delta; \Gamma \vdash_e e_i : \tau_i \quad 1 \leq i \leq m}{I; \Theta; \Delta; \Gamma; V \vdash_c \textbf{return}\, e_1 ... e_m \rightsquigarrow V; \Gamma}$$

Chamadas de funções podem ser feitas a nível de comandos. A verificação de chamadas de função se dá pelos seguintes passos:

1. Primeiro obtemos o tipo da função v, $\Theta(v) = tau_1 \mapsto, ... \mapsto \tau_m \mapsto (\tau_1, ..., \tau_p)$.
2. Se a função possuir uma restrição de tipo, verificamos se existe alguma instância de tipo ρ em I tal que quando substituimos ρ pela variável de tipo da restrição obtenha-se um tipo aplicável aos argumentos.
3. Na sequência, verificamos que os argumentos possuem o tipo exigido pela definição de função, $\Theta; \Delta; \Gamma \vdash_e e_i : \tau_i$
4. Finalmente, verificamos que as expressões passadas para os componentes da tupla de retorno devem possui o mesmo tipo dos retornos da função.

$$\frac{I; \Theta(v) = \tau_1 \mapsto, ... \mapsto \tau_m \mapsto (\tau_1, ..., \tau_p) \quad I; \Theta; \Delta; \Gamma \vdash_e e_i : \tau_i \quad I; \Theta; \Delta; \Gamma \vdash_e lv_j : \tau_j \quad 1 \leq j \leq p \quad 1 \leq i \leq m}{I; \Theta; \Delta; \Gamma; V \vdash_c v(e_1, ..., e_m)\, lv_1 ... lv_p \rightsquigarrow V; \Gamma}$$

$$\frac{\begin{array}{ccc} I; \Theta; \Delta; \Gamma \vdash_e lv_j : \tau'_j & & 1 \leq j \leq p \\ I; \Theta; \Delta; \Gamma \vdash_e e_i : \tau_i & \tau_{ir} = \tau_1 \mapsto ... \mapsto \tau_m \mapsto (\tau'_1, ..., \tau'_p) & \exists \rho.v\, \rho \in I \\ \Theta(v) = v\ a{=}{>}\tau_1 \mapsto ... \mapsto \tau_m \mapsto (\tau'_1, ..., \tau'_p) & \tau_{ir} = \textbf{subs}(\rho, \tau_1 \mapsto ... \mapsto \tau_m \mapsto (\tau'_1, ..., \tau'_p)) & 1 \leq ci \leq m \end{array}}{I; \Theta; \Delta; \Gamma \vdash_e v(e_1, ..., e_m)\, lv_1 ... lv_p \rightsquigarrow V; \Gamma}$$

4.4 Semântica de declarações

A semântica estática de declarações é dada por um julgamento $I; \Theta; \Delta \vdash_p prog \rightsquigarrow \Theta'; \Delta'; I'$ que denota que a sequência de declarações *prog* produz os contextos Θ', Δ', I' a partir de contextos Θ, Δ e I.

A primeira regra, mostra que um conjunto vazio de declarações não modifica os contextos.

$$\frac{}{I; \Theta; \Delta \vdash_p \bullet \rightsquigarrow I; \Theta; \Delta}$$

A próxima regra mostra como verificar conjuntos de declarações. A cada passo, verificamos cada uma das declarações produzindo novos contextos.

$$\frac{I;\Theta;\Delta \vdash_{def} def \rightsquigarrow I;\Theta_1;\Delta_1 \quad I;\Theta_1;\Delta_1 \vdash_p prog \rightsquigarrow I;\Theta';\Delta';}{I;\Theta;\Delta \vdash_p def\ \ prog \rightsquigarrow I;\Theta';\Delta'}$$

A validação de definição de novos tipos de dados se dá pelos seguintes passos:

1. Verificar que não há nomes de campos repetidos no tipo de dados definido;
2. Verificar que não há tipo com o mesmo nome definido previamente;

$$\frac{v \notin \mathbf{dom}(\Delta) \quad \exists! x.\exists \tau.x :: \tau \in fds}{I;\Theta;\Delta \vdash_d v\ fds \rightsquigarrow I;\Theta;\Delta'}$$

A próxima regra da semântica mostra como uma função é verificada em um programa. A validação de uma declaração de função segue os seguintes passos:

1. Verifica-se se não há declaração de outra função de mesmo nome;
2. Inicializamos o contexto Γ com os parâmetros formais da função definida, junto com o tipo desta função, para permitir chamadas recursivas.
3. Inicializamos Θ_τ com o tipo de retorno desta função.
4. Verificamos o corpo da função e validamos se todos os caminhos de execução terminam com um comando **return** com o tipo apropriado.

$$\frac{\begin{array}{ccc} v \notin \mathbf{dom}(\Theta) & \Gamma = \{(x_i,\tau_i) \,|\, 1 \leq i \leq m\} & \Theta';\Delta;\Gamma \vdash_{cs} cmds \rightsquigarrow \Gamma';V' \\ \mathbf{Seja}\ \Theta_\tau = (\tau_1,...,\tau_n) & \Theta' = \Theta, v : \tau_1, \mapsto ... \mapsto \tau_m \mapsto (\tau_1,...,\tau_n) & (\tau_1,...,\tau_n) \vdash_{ret}\ cmds \end{array}}{I;\Theta;\Delta \vdash_d v\ \ (x_1\ \tau_1,...,x_m\ \tau_m)\ \ (\tau_1,...,\tau_n)\ \ cmds \rightsquigarrow I;\Theta';\Delta}$$

As regras $(\tau_1,...,\tau_n) \vdash_{ret}\ cmds$ validam que todos os caminhos do bloco *cmds* terminam com um **return** apropriado. A primeira regra verifica que em um bloco contendo apenas um comando, este deve ser um return.

$$\frac{\Theta;\Delta;\Gamma \vdash_e e_i : \tau_i \quad 1 \leq i \leq n}{(\tau_1,...,\tau_n) \vdash_{ret}\ \mathbf{return}\, e_1,...,e_n\ \bullet}$$

Outra possibilidade válida para término de uma função é chamando outra que possua o mesmo tipo de retorno.

$$\frac{\Theta(v) = (\tau_1,...,\tau_m) \rightarrow (\tau_1,...,\tau_n)}{(\tau_1,...,\tau_n) \vdash_{ret}\ v(e_1,...,e_m)\ \bullet}$$

Em seguida, caso o último comando de um bloco seja um **if**, devemos garantir que ambos os blocos do if devem possuir um return.

$$\frac{(\tau_1, ..., \tau_n) \vdash_{ret} \ cs_1 \quad (\tau_1, ..., \tau_n) \vdash_{ret} \ cs_2}{(\tau_1, ..., \tau_n) \vdash_{ret} \ cs \, \textbf{if} \, e \, \textbf{then} \, cs_1 \, \textbf{then} \, cs_2}$$

s

Na situação do último comando de um bloco ser um **iterate**, devemos garantir que o último comando de seu bloco seja um **return**.

$$\frac{(\tau_1, ..., \tau_n) \vdash_{ret} \ cs}{(\tau_1, ..., \tau_n) \vdash_{ret} \ cs \, \textbf{iterate} \, e \, cs}$$

Caso o bloco seja formado por uma sequência de dois ou mais comandos, devemos ignorar o primeiro comando e verificar a cauda do bloco.

$$\frac{(\tau_1, ..., \tau_n) \vdash_{ret} \ cs}{(\tau_1, ..., \tau_n) \vdash_{ret} \ c \ cs}$$

As definições de classes são verifcadas da seguinte forma: Para cada funcão f da classe idetificada por v_{cls} iserimos o tipo da fuçãi $f{::}\tau$ com uma restrição $v_{class} \ a$ no contexto de funções θ.

$$\frac{I; \Theta; \Delta \vdash_{clsd-a} ps \rightsquigarrow I'; \Theta'; \Delta}{I; \Theta; \Delta \vdash_{cls} class \ v \ a \ ps \rightsquigarrow I'; \Theta'; \Delta}$$

$$\frac{}{I; \Theta; \Delta \vdash_{clsd-a} \bullet \rightsquigarrow I; \Theta; \Delta}$$

$$\frac{I; \Theta; \Delta \vdash_d v \ \tau_1 \mapsto \tau_n \ \rightsquigarrow I; \Theta'; \Delta \quad \begin{array}{c} \Theta' = \Theta, v \ :: \ a => \tau \\ a \text{ ocorre livre em } \tau \end{array}}{I; \Theta; \Delta \vdash_{clsd-a} v \ :: \ ts \, ps \rightsquigarrow I'; \Theta'; \Delta}$$

As definições de isntâcias de classes de tipos I seguem os mesmo princípios das declarações de funções. Exceto pelo fato que elas não insere as definições das funções da instêancia em Θ, mas apenas registram a definição da classe em I.

$$\frac{I; \Theta; \Delta \vdash_d fs \rightsquigarrow I'; \Theta'; \Delta}{I; \Theta; \Delta \vdash_{insd} \textbf{instance} \ v \ \rho \ fs \rightsquigarrow I' \cup (v \ \rho); \Theta; \Delta}$$

4.5 Semântica de programas completos

Programas Lang são formados por uma ou mais declarações. Dizemos que um programa Lang é bem formado se este possui pelo menos uma função de nome **main** de tipo $Void$, em que o arranjo de strings representa possíveis argumentos de linha de comando fornecidos ao programa.

A regra a seguir faz essa validação, inicializando o contexto Θ com as definições iniciais presentes em Θ_0:

$$\frac{\Theta_0; \bullet \vdash_p prog \rightsquigarrow \Theta; \Delta \quad \Theta(\mathbf{main}) = \mathbf{Void}}{\vdash_{wf} \ prog}$$

Com isso, finalizamos a especificação da semântica estática de Lang. Na próxima seção apresentaremos a semântica dinâmica de programas Lang.

5 Definição Formal da Sintaxe de $lang^2$

A Gramática 2 apresenta a gramática livre-de-contexto que descreve a sintaxe da linguagem $lang^2$ usando a notação EBNF. Os símbolos terminais serão grafados como **'sym'** e as palavras reservadas como **palavra**. Um símbolo com a grafia de *TOKEN* significa que esse símbolo denota um conjunto de lexemas que denotam o mesmo token, como o caso de identificadores e números inteiros, por exemplo. A forma { x } denota a repetição de 0 ou mais vezes de x e a forma [x] denota que x é opcional.

prog	→	{decl}
decl	→	data | func | classDec | instDec
data	→	**data** *TYID* **'{'** {bind} **'}'**
bind	→	*ID* **'::'** typeAnnot **';'**
func	→	*ID* {*ID* } **'::'** typeAnnot block
classDec	→	**class** *TYID* *ID* **'{'** {bind} **'}'**
instDec	→	**instance** *TYID* **for** *ID* **'{'** {func} **'}'**
typeAnnot	→	tyJoin
	|	type **'->'** typeAnnot
tyJoin	→	type
	|	type { **'&'** type}
type	→	type **'['** **']'** | btype
btype	→	**Int** | **Char** | **Bool** | **Float** | *TYID* | **Void**
block	→	**'{'** {cmd} **'}'**
stmtBlock	→	block | cmd
cmd	→	**if** **'('** exp **')'** stmtBlock
	|	**if** **'('** exp **')'** stmtBlock **else** stmtBlock
	|	**iterate** **'('** loopCond **')'** stmtBlock
	|	**return** {exp} **';'**
	|	lvalue **'='** exp **';'**
	|	*ID* **'('** [exps] **')'** [**'<'** lvalue {**','** lvalue} **'>'**] **';'**
loopCond	→	*ID* **':'** exp
	|	exp
exp	→	exp operator exp
	|	**'!'** exp | **'-'** exp
	|	**true** | **false** | **null**
	|	*INT* | *FLOAT* | *CHAR*
	|	**new** type [**'['** exp **']'**]
	|	*ID* **'('** [exps] **')'** **'['** exp **']'**
	|	lvalue
	|	**'('** exp **')'**
operator	→	**'&&'** | **'=='** | **'!='** | **'&&'** | **'+'** | **'-'** | **'*'**
	|	**'/'** | **'%'** | **'<'**
lvalue	→	lvalue **'.'** lvalue
	|	**'['** exp **']'**
	|	*ID*
exps	→	exp {**','** exp }

Gramática 2: Sintaxe da linguagem $lang^2$